SESSION SUMMARY

Introduction to Claude Code with Antigravity

Day One Intermediate Sprint

Mentor

KVS Dileep Head of Generative AI Education, Outskill

Host

Kartik Program Manager, Outskill

Series

AI Masterclasses & Sprints by Outskill

ABOUT THIS DOCUMENT

A cleaned, hierarchical reconstruction of the live session. The transcript has been organised by topic, freed of timestamps, and rewritten for clarity while preserving the mentor's reasoning, examples, and live-build moments.

Framing the Sprint

The session opens with a clear statement of intent. This is not a beginner workshop, and it is not a slow-paced walkthrough where every button is pointed out. It is an intermediate sprint that sits deliberately above the comfort floor of the regular cohort programs, designed for learners who have

already built foundational fluency with AI tools and now want to push into the territory of building with them.

The mentor is upfront about three constraints that shape the next two hours. First, there will be no slides. The teaching surface is a whiteboard and a live screen. Because the session is about building, anything pre-rendered would only get in the way. Second, the chat is restricted to hosts and panelists. Some learners want quiet focus, others want conversation, and trying to please everyone fragments attention; the trade-off is a calmer learning environment at the cost of less back-and-forth. Third, there is no hand-holding. A few major doubts will be addressed; the rest is left as an exploration assignment for the learner after the recording is available.

Why the sprint format exists

Outskill's core programs cater to a wide audience and necessarily move at a pace that keeps everyone aboard. A persistent piece of feedback from the community has been a hunger for slightly more advanced material the kind of content that would otherwise leave the median learner behind in a regular cohort. The sprints are built precisely to scratch that itch. They are spaces where the mentor can move faster, assume more, and skip the introductory scaffolding without apology.

That comes with a cost the mentor wants the audience to accept upfront. Some sections will land cleanly. Other sections will fly past, and that is acceptable. The recording exists for a reason. Anxiety about keeping up is the wrong response; the right response is to watch with intent and revisit the recording for whatever didn't land the first time.

Today versus tomorrow

Today's session is intermediate. Tomorrow's session is advanced. Today emphasises taste, judgment, and the systems thinking that surrounds a build; the actual mechanics of clicking through a UI are largely treated as the learner's responsibility. Tomorrow will go deeper into the mentor's personal advanced setup the harness of skills, memory tools, and orchestration patterns he uses every day.

THE PROMISE OF THE SESSION

Two parts. Part one: basic setup, fundamentals, and a small build. Part two: a harness, a system, for building applications the mentor's own workflow, demonstrated live on a real project he has not previously attempted.

The Bigger Picture: Taste, Systems Thinking, Craftsmanship

Before any tool was opened, the mentor offered a piece of scaffolding that he had been ruminating on for several days. It had been triggered by the release of Claude Mythos and a public observation by a respected AI security researcher: Mythos was finding bugs in days that the researcher had been unable to find across an entire career. One of those bugs was inside OpenBSD an operating system widely considered among the most hardened in existence, used in serious defence applications. A bug that had eluded human eyes for many years was surfaced by the model in a fraction of the time.

That moment forced an uncomfortable question: what is the plan B if AI replaces me? It is no longer a science-fiction concern. The mentor referenced a recent Indian operation that pays workers to wear head-mounted cameras while doing their daily jobs, generating training data for Chinese robotics companies. The frontier of imitation has moved into the physical world. The pressure is not theoretical.

In response, three qualities emerged as the mentor's personal shortlist of what continues to matter. They are presented as opinions, not laws but they shape the rest of the session, because every choice the mentor makes during the live build is justified through one of these three lenses.

Taste

AI can produce a hundred options in ten seconds. Taste is the accumulated judgment that lets a person look at those hundred options and say, this one. Not because it is provably best, but because some sense developed over thousands of similar choices recognises it as right. Taste is hard to teach directly and harder to fake. It is also what stops a builder from drowning in the sheer volume of AI-generated possibility.

Systems Thinking

Systems thinking is knowing what matters. It is the ability to step back from a problem and ask whether it is even worth solving, whether the framing is right, whether the chosen approach lines up with the goal. AI can execute tasks at a speed humans cannot match, but it does not, on its own, decide which tasks are worth executing. That decision is the human contribution.

Craftsmanship

Craftsmanship is the depth of mastery that AI cannot keep up with, achieved by going further into a discipline than most people are willing to. The path is long and uncomfortable, but it is durable. A craftsman who has spent years inside a domain notices the texture of a problem in ways that surface-level systems will miss for a long time.

HOW THIS SHAPES THE SESSION

Taste and judgment will be the dominant focus today.

Craftsmanship will receive lighter touches throughout.

Mechanics which button to click, which menu to open are intentionally underspecified, because the mentor wants learners to develop their own muscle for figuring those things out.

The Toolset: Claude Code and Antigravity

The session title pairs Claude Code with Antigravity, but the mentor is candid about the relationship between the two. Claude Code is the centrepiece. Antigravity is a supporting layer. Going deeper into Antigravity itself is reserved for a future standalone sprint.

The choice of Claude Code as the lead tool is not arbitrary. Anthropic's annualised revenue has reached around the thirty-billion-dollar mark, drawing very close to OpenAI's run rate. A significant share of that catch-up has come from Claude Code, which started as a small internal experiment by a tight group inside Anthropic and grew into one of the most influential developer tools currently shipping. The mentor's own daily workflow is built around it, which is why he is teaching it first.

Roles in the workflow

Claude Code	Antigravity
The agent. The brain and muscle that does the work.	The integrated development environment. The window through which files and folders are visible.
Executes terminal commands, writes code, makes edits, manages packages, handles git operations.	Lets the human inspect what was built, navigate the project, and view hidden files like the .claude folder.
Runs in the terminal in its purest form, or in the editor's extension panel.	Built by Google. A close cousin of VS Code under the hood, with an additional Agent Manager mode.
Subject to token usage limits tied to the Anthropic plan.	Acts as a backup execution surface when Claude Code tokens are exhausted.

Why this pairing, not another

Antigravity is interchangeable with VS Code, Cursor, Windsurf, Kiro, or any other VS Code-derived editor. The mentor specifically avoids long debates about which IDE is best, because at this stage the underlying mechanics are nearly identical. Pick one, install the Claude Code extension from Anthropic, and the workflow is the same.

The reason Antigravity earns the spot is twofold. First, it is built by a serious team and ships with sensible defaults. Second, and more importantly, it has its own Agent Manager mode powered by Gemini under the hood. That second engine becomes a lifeline when Claude Code's tokens run out mid-build; the project can be picked up by Antigravity's agent and pushed forward until the Claude tokens reset.

THE MENTAL MODEL

Claude Code is the executor. Antigravity is the editor and the backup. The two together form a system that is more resilient than either alone.

Other access paths to Claude Code

Learners who are uncomfortable with the terminal can install Claude Desktop, where a Code tab provides a similar experience inside a more familiar shell. The mentor does not demonstrate this path during the session, but he flags it as a valid alternative. The terminal remains his preferred surface because it exposes Claude Code at full capability with the least amount of UI mediation.

Terminal Foundations

Before any building begins, the mentor takes a deliberate detour into terminal basics. The detour is short but non-negotiable. A person who does not understand what the terminal is doing will eventually be unable to interpret what Claude Code reports back, and that loss of interpretive ability will quietly cap how far they can take any project.

The recommended resource is codingformarketers.com, a site the mentor has come to like because it teaches the bare essentials in a non-coder-friendly register. He explicitly tells the audience to spend at least one hour on the site before they ever open Claude Code for the first time.

The only command that truly matters

There are exactly three navigation commands a learner needs to understand: change directory, list files, and print the working directory. Of these, only one needs to be actively used.

- **cd** change directory. The single command a builder will run repeatedly to point the terminal at the right project folder. This is the one to internalise.
- **ls** list files. Useful to know about, but rarely needed in practice; the editor view does this job better.
- **pwd** print working directory. Shows the current location. Helpful as a sanity check but again not a daily-use command.

Platform differences

On macOS or Linux the built-in Terminal application is sufficient. On Windows the recommended tool is PowerShell, which is a separate piece of software from the older Command Prompt and offers a more capable shell for the kind of work Claude Code expects. Installing PowerShell is the first concrete step a Windows learner should take.

Other terminal categories let the agent handle them

There are four broad categories of terminal commands a builder will encounter: navigation, file operations, package management, and source control through Git. Of these, only navigation requires manual fluency. The other three categories are handled by Claude Code itself, which knows how to install packages, create files, run Git commands, and recover from common errors. The learner's job is to understand what those commands mean when Claude Code reports them back, not to type them by hand.

THE DIRECTORY HYGIENE RULE

Every new project gets its own folder. This is a discipline, not a suggestion. A clean folder gives the agent a clean working surface, a clear scope for skills and configuration, and a safe boundary for any local state the project generates.

Environment variables and API keys

The other section of the recommended resource that learners must internalise covers environment variables. Secrets API keys, passwords, anything that would let an attacker impersonate the project must live in a dedicated environment file and must never be committed to a public repository. The .gitignore file is the safety net that keeps this rule enforceable, and the mentor returns to it explicitly during the live build later in the session.

Setting Up the Environment

With the framing complete, the live build begins. The first concrete task is to set up the workspace for a new project the mentor has chosen on the fly: a Model Console application, modelled on a recent Perplexity feature where multiple AI models debate a single prompt and converge on a synthesised answer. Crucially, the mentor has not built this application before. The audience is watching not a polished demo but an honest first attempt.

Creating the project folder

A new folder named Model Console is created inside Downloads. The choice of location is incidental; the discipline of giving the project its own folder is not. With the folder in place, Antigravity is launched and pointed at it. The file tree on the left shows an empty directory, which is exactly the right starting state. The trust prompt that Antigravity raises when opening a new folder is accepted.

Installing the Claude Code extension

Claude Code is added to Antigravity as an extension. The mentor demonstrates this through the extensions panel on the left side of the editor and stresses one specific point: install the official extension published by Anthropic. The package identifier visible in the listing `anthropic.claude-code` is the verification that the right one has been selected. Third-party look-alikes exist and should be avoided.

Once installed, the extension is accessible from either the top bar or the left rail of the editor. Opening it reveals a fresh Claude Code session with no previous context, which is the cleanest possible starting state for a new project.

Choosing where Claude Code lives

Two surfaces are available: the native Claude Code panel inside the editor, and a terminal session running Claude Code as a CLI. The mentor begins in the panel for clarity but quickly switches to the terminal, because the terminal exposes more capability and shows usage information that the panel hides. The terminal also makes it easier to run the slash commands that are central to the workflow.

WHY THE TERMINAL WINS LONG-TERM

The native panel is friendlier on day one, but the terminal gives faster access to settings, more visible token usage, and a less mediated relationship with the agent. The mentor recommends migrating to it as soon as the initial discomfort subsides.

Account requirements

Claude Code is not available on the free Anthropic plan. Access requires either a Pro or Max subscription, which routes Claude Code through the user's existing plan-level usage; or, alternatively, an API setup configured at console.anthropic.com with prepaid credits. The mentor is on the Max plan throughout the session and notes that even there, sustained heavy use will push token usage uncomfortably close to the cap.

Essential Claude Code Commands

Claude Code is operated through slash commands, and a small handful of them carry most of the weight in a real workflow. The mentor singles out three as the indispensable starting set.

/login

The first command in any new session. It connects Claude Code to the user's Anthropic account so usage flows against the right plan. For Pro and Max subscribers this is a one-step authentication. For everyone else it routes through the API console where credits must be added before any work can be done.

/usage

The mentor calls this the most underrated command in the toolkit. Account and usage information is surfaced through this command, showing both the percentage consumed in the current session and the running totals against the weekly limits. He demonstrates that even a clean session, freshly logged in, sits around twenty per cent of a weekly Sonnet allowance once a few back-and-forth turns have happened. The lesson: usage is a finite resource that disappears faster than instinct suggests, and watching it is a habit worth building.

PRO PLAN REALITY CHECK

On the Pro plan, an active building session can exhaust available tokens in roughly twenty to twenty-five minutes of intensive coding. Plan accordingly. The Max plan extends the runway considerably but is not infinite either.

/model

The third critical command. It opens the model picker and lets the user switch between the available Claude variants Opus, Sonnet, and Haiku among others. The choice of model has direct consequences for both the quality of output and the rate at which tokens disappear, which leads naturally into the next section.

Other commands worth knowing

- **/skills** lists all skills currently available to the agent.
- **/mcp** surfaces the Model Context Protocol servers the agent can call. The mentor's own setup includes Airtable, Context7, Asana, Canva, Gmail, Google services, Granola, Miro, Notion and others.
- **/plugin** manages plugin installations from the marketplace. A plugin is a bundle of related skills.

Choosing Your Model: Opus versus Sonnet

Once the model picker is understood, the question becomes when to use which. The mentor's rule is explicit, repeated more than once during the session, and worth treating as a default until experience teaches otherwise.

THE MODEL SELECTION RULE

Use Opus for the heavy thinking and the hard planning.

Once the plan is locked, switch to Sonnet (or another lighter model) for execution.

Using Opus for everything will burn through tokens in ten to fifteen minutes of work and leave the project stranded.

Why this split makes sense

Opus is the deepest reasoner in the family. When the task is genuinely a thinking task drafting a product requirements document, choosing an architecture, debugging a stubborn issue, deciding how a feature should behave the marginal quality gain from Opus is worth its higher cost. Once the thinking is done and the work descends into mechanical implementation, that quality premium is mostly wasted, and Sonnet does the same job at a fraction of the budget.

The discipline is to be honest with oneself about which mode the current task belongs in. There is a temptation to leave the model on Opus because it feels safer, but that habit converts a one-week token budget into a ninety-minute session.

Switching mid-build

During the live build, the mentor leaves the model on Opus longer than he would in a normal workflow because of an intermittent Claude outage that disrupts the session. He flags this explicitly: under normal

conditions he would have switched to Sonnet much earlier in the implementation phase to preserve runway.

The Chef Analogy: Skills, MCPs, and the Agent

Having explained the surface mechanics, the mentor introduces the conceptual model that holds the rest of the session together. It is a kitchen analogy, and it is one of the most useful framings in the entire teaching.

The three roles

Element	Kitchen Equivalent
Skills	Recipes repeatable playbooks written by people who have already solved a problem well.
MCP servers	Tools the knives, pans, and ovens that turn a recipe into food.
The agent (Claude Code)	The chef the one who reads the recipe, picks up the tools, and produces the dish.

What this framing buys you

The analogy is not decorative. It does real work in the mentor's reasoning. When a problem appears, the question becomes: which recipe applies here? Is there a recipe at all? If not, the chef has to improvise, and improvisation is more expensive than execution. If yes, do we have the right tools to follow that recipe, or are we missing an MCP that the recipe depends on?

This decomposition also clarifies why a learner cannot just throw a vague request at Claude Code and expect a great result. The chef can only work as well as the recipe and the tools allow. Curating the recipes curating the skills is therefore one of the highest-leverage things a builder can do.

THE KNOWLEDGE GAP PROBLEM

Most builders are not the best product manager in the world, not the best engineering manager in the world, and not the best designer in the world. Yet they still want to ship products that shine. Skills exist to close that gap by importing the playbooks of people who are the best at those roles.

Standing on the shoulders of giants

The mentor's grandmother analogy lands the philosophical point. If a recipe handed down by a grandmother already exists, the wise move is to use it and add a personal flourish. The stubborn move is to insist on rediscovering the recipe alone, from first principles, and producing something inferior in the process. The same principle applies to building software with AI. There are people who have already worked out how to write great PRDs, run great product discovery, and execute well their playbooks are public, and refusing to use them is a kind of pride that costs the builder time and quality.

The PM Skills Repository

To demonstrate the recipe-curation idea concretely, the mentor opens a public GitHub repository called PM Skills, maintained by a product manager named Pavel Hurin. The repository is described as an AI operating system for better product decisions, and it contains skills derived from the practices of well-known names in the product management world Marty Cagan, Teresa Torres, and others.

Judging the source

Before installing anything, the mentor walks through how he decides whether a public repository is worth borrowing from. The signals he relies on are simple but informative.

- **Forks:** around 1,100 people have copied the repository to add their own modifications. Forks indicate active builders who found the work valuable enough to extend.
- **Stars:** around 10,000 people have starred it. Stars are a softer signal but still meaningful they show breadth of interest.
- **Source quality:** the playbooks are explicitly attributed to recognised practitioners, not anonymous contributions.

THE TASTE-DRIVEN SHORTCUT FOR INSTALLING SKILLS

Do not install every skill in a repository by default.

Paste the repository link into Claude Code and describe what you are trying to build.

Ask the agent which skills are actually relevant to that build, and install only those.

This keeps the working surface clean and prevents unrelated skills from interfering with the project.

The live demonstration

The mentor pastes the PM Skills repository URL into Claude Code along with a short description of the Model Console application a web app where a user submits one prompt and three or four chosen models debate it, work with each other, and converge on a final answer accompanied by an agreement-and-disagreement table. He then asks the agent which skills from the repository are actually useful for this particular build.

The agent's response is candid and useful. Most of the skills in the repository are aimed at strategic product work, not at building an application. For a model-debate tool, only a handful are genuinely relevant. The agent recommends a must-have set and a nice-to-have set, suggests the marketplace and install commands needed to set them up, and explicitly advises skipping the rest. The mentor accepts the recommendation and runs the install commands.

Each install command asks how widely the skill should be available across all repositories, across the team, or only inside the current repository. The mentor chooses the local-only option deliberately. His main setup already has a different and curated skill arrangement, and he does not want this demonstration to pollute it. After the installs complete, a /reload-plugins command brings everything online.

What plugins really are

During this section the mentor offers a clean definition. A plugin is simply a collection of related skills bundled together. A marketing plugin might contain skills for SEO, copywriting, and landing-page design; a product management plugin might contain skills for discovery, PRD authoring, and execution planning. The packaging is a convenience for installation; the underlying units are the individual skills.

Building the Model Console: From Idea to PRD

With the skills installed, the mentor begins the actual build. The opening move is not to write any code. It is to author a product requirements document, because the skills that are now loaded include a discovery and PRD-creation workflow built by serious product managers, and the cheapest place to fix a bad idea is before any line of code exists.

The brainstorm prompt

The mentor asks Claude Code to start building the Model Console using the newly installed skills. The agent, recognising the context, does not jump straight into PRD generation. It runs a short brainstorm first, asking the questions any decent product manager would ask before committing to a specification.

Questions the agent surfaces

- Models which three or four models should sit on the council?
- Debate mechanics how many rounds? Independent answers in round one, critique and update in round two, optional final statements in round three?
- Synthesizer should a separate judge model write the final verdict?
- Form factor web app, plugin, skill, something else?
- API access which provider, which keys?
- Output format table, prose, exportable artefact?

The mentor's answers

- **Models:** Claude Sonnet 4.6, GPT-5, the latest Gemini, and the latest Grok variant.
- **Rounds:** two mandatory rounds, with an optional third round that auto-fires.
- **Synthesizer:** Claude Opus 4.6 writes the final verdict.
- **Form factor:** a web app where the prompt is submitted, debate rounds are visible as they unfold, and the output appears as a verdict table with confidence scores, a reasoning trace, and an exportable markdown version.
- **API access:** OpenRouter, which exposes all four target models through a single OpenAI-compatible interface.

Catching a hallucination early

The agent flags two of the requested models as not existing yet a Gemini version that has not been released and a Grok variant that is also unavailable. Rather than accept the agent's word at face value, the mentor opens OpenRouter directly to verify. He searches the model catalogue, finds the actually-available identifiers, and feeds those back into the conversation. This is taste in action: trust but verify, especially when the agent claims something does not exist.

THE VERIFICATION REFLEX

When an agent says a model, library, or service does not exist, do not take it on trust. Open the source and check. Models are released constantly, agents have stale catalogues, and a wrong rejection here will cost more than the thirty seconds it takes to confirm.

PRD generation

Once the parameters are settled, the agent begins writing the PRD. The mentor uses the build time as a natural break point in the session. When work resumes, the document is ready and visible in the file tree

of Antigravity which is exactly why Antigravity is in the workflow at all. Without an editor showing the live file system, the build would be invisible.

The PRD that emerges is comprehensive. It opens with a concise summary: a web application that sends a single user prompt to four models, has them debate each other, and uses Claude 4.6 as a synthesizer to produce a final verdict with a side-by-side comparison table. It captures the context that the user otherwise has to open four browser tabs, paste the same prompt into each, and mentally compare the outputs, a process the document fairly characterises as slow, lossy, and lacking structured debate.

It then enumerates key results, target market segment, value proposition, gains, a sketched user experience flow with a text wireframe, the verdict table layout, and the technology choices. The mentor walks through it visually and pronounces it acceptable. Not perfect, but substantively right and more thorough than what he would have produced unaided in the same time.

Adding a Design System

A PRD that does not address visual design will produce a functionally correct application that nobody enjoys using. The mentor's next move is to bolt a design system onto the document before any code is generated, because retrofitting design after the fact is significantly more painful than baking it in upfront.

SuperDesign

The mentor's go-to tool here is superdesign.dev, a free resource that surfaces curated design systems and exposes them as drop-in prompts. He browses the catalogue looking specifically for something clean and minimal taste again and settles on a system that fits the aesthetic he wants for the Model Console.

The chosen design system has a Copy Prompt button. He copies the prompt, returns to Claude Code, and asks the agent to add these design considerations to the PRD. The agent updates the document with a full design system section: typography, palette, spacing, component conventions, and the visual rules the build will follow.

WHY UPSTREAM DESIGN BEATS DOWNSTREAM DESIGN

Design decisions made before code is written propagate cleanly through the build. Design decisions retrofitted into a working app force rewrites of components that were already shaped by the wrong assumptions. A few minutes spent on a SuperDesign prompt at the PRD stage saves hours later.

When Claude Goes Down: Switching to Antigravity Agent Manager

Just as the build is set to begin in earnest, Claude experiences a service disruption. Sessions stop responding, login attempts hang, and the mentor confirms by checking that other users worldwide are seeing the same problem. This is the moment the dual-engine setup justifies itself.

The graceful failover

Antigravity's Agent Manager is opened. Because Antigravity ships with its own agent powered by Gemini, the project can continue to move forward even with Claude Code unavailable. The mentor opens the Model Console folder inside the Agent Manager and asks it to read the existing files and catch up on what is happening.

The Agent Manager parses the PRD, recognises the structure, and is ready to continue from the same point. The mentor uses an interesting touch here: he tells the Agent Manager which skills he had been using inside Claude Code, so that the new agent can match the working style as closely as possible. Skills installed inside the project folder are accessible to whichever agent reads them, but flagging the intention helps the new agent pick up the rhythm.

THE LESSON FROM THE OUTAGE

Build your workflow so that a single point of failure does not stop the project. A second agent attached to the same project folder is not redundant; it is insurance. When the primary fails, the work continues.

Claude returns

Partway through the failover, Claude Code comes back online. The mentor halts the Agent Manager mid-flight and resumes inside Claude Code, since it remains his preferred surface. The brief interruption demonstrates that the failover works without committing to it as the day's main path.

A useful conversational command

During the recovery, the mentor demonstrates a simple but underused trick: while the agent is working, a status question can be asked in plain language. "What is happening?" returns a clean readout of where the agent is in the current task what is done, what is in progress, what is queued, what assumptions or risks are being tracked. It is a low-effort way to keep the human in the loop without breaking the agent's flow.

Environment Variables and API Keys

With the agents working again, the build reaches the point where an OpenRouter API key is needed. This triggers the most security-sensitive moment of the session, and the mentor handles it with deliberate care.

Getting the key into the project safely

Rather than paste the API key into the chat which would store it in conversation history and expose it to anyone who later sees the log the mentor instructs the agent to create a .env file and tells it that he will paste the key in himself. The agent acknowledges this and creates the file, leaving the value blank for the user to fill.

THE CARDINAL RULE

Never paste API keys into chat windows. Always use a .env file. Always confirm that .env appears in .gitignore before any commit happens.

The screen-sharing pause

Before pasting the actual key into the .env file, the mentor stops sharing his screen. He then asks the audience to guess why. The answer is that an API key visible on a streamed screen is an API key that may have been seen by hundreds of viewers, any one of whom could screenshot it. Once shared, a key must be considered compromised. He generates a fresh key in OpenRouter, pastes it into the local .env file, saves it, and then resumes screen sharing.

The .gitignore check

Before any push to GitHub, he opens .gitignore and verifies that .env files are listed there. They are. The agent had the good sense to set this up correctly during scaffolding, but the mentor's habit of checking anyway is the right pattern to follow even when the agent's defaults are sensible. Trusting the agent on security is acceptable; verifying the agent on security is mandatory.

Running the Dev Server and the First Demo

With the API key in place, the mentor asks the agent to start the dev server. The build is now in its moment-of-truth phase. The local URL opens in a browser and the Model Council app appears: a clean interface with a single prompt field, a clear branding line "one prompt, four models, structured debate, one final verdict, synthesised by Claude 4.6" and a call-to-action button labelled Convene the Council.

The first prompt

The mentor types a deliberately self-referential question is Claude Code better than Antigravity and clicks Convene the Council. The four models begin responding.

Round one: independent answers

- **Claude Sonnet** argues that the two are not really comparable because they serve different purposes.
- **Gemini** frames the comparison as essentially comparing apples to a physics-defying concept, suggesting it has interpreted Antigravity literally.
- **Grok 4.2** delivers a confident verdict that Claude Code is significantly better than Antigravity.
- **GPT-5** responds slowly but produces a short structured answer.

Round two: critique and update

Each model now sees the others' answers and gets a chance to revise. The mentor's UI shows the agreements and disagreements clearly, with each model marked against the others. The debate mechanic works as designed.

Round three: final statements

In the optional final round, three of the models converge on the same conclusion Claude Code is the right answer for software work while GPT-5 wanders down a tangent. The synthesiser produces a verdict table summarising agreements, disagreements, confidence levels, and a reasoning trace.

Catching the hallucination

The verdict table reveals something important. Claude Sonnet and Gemini argue that Antigravity is not a real software tool, while Grok and GPT-5 treat it as a plausible product and produce substantive comparisons. This is a genuine factual disagreement among the council members about whether the thing being asked about exists. Two models hallucinate Antigravity-the-product; two refuse to. The synthesiser flags the disagreement honestly.

The mentor takes this in stride. The hallucination is a real-world failure mode, not a defect in the build. It also points to the next obvious improvement: the models need access to current information.

Adding Web Search via OpenRouter

The hallucination problem has a clean solution. The mentor asks the agent for advice on adding a web search capability and confirms his preferred provider would be the cheapest viable option.

The OpenRouter trick

The agent surfaces a particularly elegant answer: OpenRouter has built-in web search baked into every model. Appending the suffix :online to a model slug routes the request through a search-augmented version of the same model, with no additional infrastructure or separate API to manage. Alternative providers exist EXA, Tavily, BraveSearch among others but the OpenRouter path is the lowest-friction option for a project that is already using OpenRouter as its model gateway.

WHY THIS MATTERS

Reducing hallucination is often less about prompt engineering and more about giving the model access to current ground truth. A web search hookup is one of the highest-leverage improvements a builder can make, and it is increasingly available as a one-line addition rather than a separate integration project.

The second demo

With web search enabled, the mentor reruns the same prompt. The model responses now reference current information about both Claude Code and Antigravity as actual tools. The hallucination disappears. The verdict table now reflects a real comparison: agreement areas, disagreement areas, unique discoveries surfaced by individual models, and a more comprehensive synthesis.

The mentor pauses to make a broader point. Perplexity charges roughly two hundred dollars per month for a comparable feature in their max plan. The fact that a working version of the same idea can be built in an evening, for a fraction of the cost, is the whole point of being a builder. The frontier of what used to require a paid product is now reachable with a focused PRD, the right skills, and a few good tool choices.

Pushing to GitHub Safely

With the application working, the mentor walks through the process of pushing the code to GitHub. The instruction he gives the agent is short but contains an important constraint.

THE PUSH INSTRUCTION

Put this repository on GitHub.

Do not commit the PM Skills.

Write a good README.

Why exclude the skills

The PM Skills are already publicly available in their original repository. Re-committing them inside the Model Console project would bloat the repository, muddy the attribution, and serve no purpose. The agent understands this and excludes them via .gitignore additions.

The .gitignore audit, again

Before the push proceeds, the mentor opens .gitignore one more time to confirm that the .env file is excluded and that no secret material is about to be uploaded. The check is fast a few seconds and the discipline of doing it every time is the difference between a private project and a leaked credential.

The connection to GitHub

If the user already has a GitHub repository to push to, the simplest path is to give Claude Code the repository URL and ask it to connect. The agent handles authentication, remote configuration, and the initial push. There is no need to remember the underlying git commands; the agent runs them and reports the results.

After the push, the project is public on GitHub. The README the agent generated describes the application, the council mechanics, the synthesiser, and how the debate is structured. The repository is ready for anyone to fork, extend, or build on top of.

Deployment as the next step

The mentor flags but does not execute the deployment step. Vercel and Netlify will both happily deploy a GitHub repository with minimal configuration. The reason he stops short of doing it live is purely token-economic: the deployment process would consume more of his Claude allocation than he wants to spend in the demo. The path is open; it just is not walked today.

Token Economics and Usage Discipline

Throughout the session, the mentor returns to the question of token usage. It is not a peripheral topic; it is the constraint that shapes nearly every other decision a builder makes.

What the live numbers showed

Metric	Reading at end of session
Current session context used	Approximately 28 percent.
Weekly all-models usage	Approximately 11 percent.
Plan	Max plan ($200 tier).
Estimated additional features the remaining session budget would support	Two to three more features before the session would have to end.
OpenRouter spend during the build	Approximately 26 cents from $9.67 down to $9.41.

The clarity-first principle

The single biggest determinant of token efficiency is not the model choice, the prompt phrasing, or the IDE. It is the clarity the human brings to the task. A builder who knows what they want, in concrete terms, will produce the same outcome at a fraction of the token cost of a builder who is figuring it out as they go. Most token waste is the agent doing extra exploratory work because the human did not lock down the specification first.

THE MANTRA

If you have clarity, you save tokens. If you do not have clarity, your tokens will be exhausted before the project is built. Plan in your head before you plan in the agent.

Subscription versus API

Asked which path to recommend, the mentor leans toward the subscription. The Pro and Max plans bundle a meaningful amount of usage at a flat rate, which is usually more economical for a builder doing intensive work than the metered API. The API has its place, especially for production deployments, but for a learner building during the day the subscription is the saner default.

Watching usage actively

The /usage command is not a once-a-week check. It is a habit to be developed during a session. The bottom of the terminal also displays a percentage of current-session context, which is the most immediate signal that a session is heating up. Treat that number the way a driver treats a fuel gauge.

The Workflow, End to End

Strip away the live demonstration and the workflow the mentor showed reduces to a clean sequence. Each step has a purpose; each step compounds with the others.

Stage one preparation

- Decide what you are building. Write it down for yourself in a sentence or two before you open any tool.
- Create a dedicated project folder. Open it in Antigravity (or your editor of choice). Trust the workspace.
- Install the official Claude Code extension from Anthropic. Avoid third-party clones.
- Open Claude Code, log in, and check usage. Set the model to Opus for the planning phase.

Stage two recipes and tools

- Identify a public repository of skills written by people more expert than you. Judge it by forks, stars, and named contributors.
- Paste the repository URL into Claude Code with a description of what you want to build. Ask which skills are actually relevant.
- Install only the recommended skills, scoped to the local project unless you have a good reason to install globally.
- Reload plugins and confirm with /skills that the tools are available.

Stage three specification

- Ask the agent to begin building. Let it run a brainstorm phase before any PRD is written.
- Answer the brainstorm questions with concrete choices: models, mechanics, form factor, output format, API providers.
- Verify any factual claims the agent makes especially claims that something does not exist.
- Have the agent generate the PRD. Read it carefully in the editor's file view.

Stage four design

- Pick a design system from a curated source like SuperDesign. Choose by taste and by fit with the application's purpose.
- Copy the design prompt and ask the agent to merge it into the PRD.

Stage five build

- Switch to Sonnet for the implementation phase to preserve token budget.
- Have the agent scaffold the project, generate the code, and create a .env file. Paste secret keys yourself; never let them appear in chat.
- Verify .gitignore excludes .env before any commit.
- Start the dev server. Open the application. Test the core flow.

Stage six improve and ship

- Identify the most obvious failure mode of the first version. For the Model Console, this was hallucination, fixed by adding web search.
- Make a single targeted improvement and test again.
- Push to GitHub with a good README, excluding any vendored skills or local state.
- Optionally deploy via Vercel or Netlify, which read the GitHub repository directly.

Stage seven recover and continue

- If Claude Code becomes unavailable, switch to Antigravity's Agent Manager and continue against the same project folder.
- If tokens are exhausted, pause until the weekly reset, or switch agents, or downgrade the model in use.
- If the agent gets stuck, ask it in plain language what is happening. Use that readout to decide whether to redirect or wait.

Common Pitfalls and How to Avoid Them

Several traps appeared either explicitly or implicitly during the session. They are worth collecting in one place because they account for most of the avoidable failures a new Claude Code user runs into.

Using Opus for everything

The fastest way to exhaust a token budget is to leave the model on Opus through an entire build. Opus is meant for the thinking phases. Once the plan is locked, switch down. The mentor stresses this rule more than once for a reason.

Installing every skill in a repository

Skills that are not relevant to the current project still occupy context, can be invoked unexpectedly, and clutter the working surface. Always ask the agent which skills are actually useful for the build at hand and install only those.

Pasting API keys into chat

This is the single most preventable security mistake. Always use a .env file. Always have the agent set up the file structure first, then paste the actual key value yourself, into the file, never into the conversation.

Forgetting to verify .gitignore

A .env file that is not in .gitignore will be pushed to GitHub. Once a key is in a public repository, it is compromised assume it has been scraped within minutes, even if rotated immediately afterwards. Verify .gitignore manually before any commit.

Trusting agent claims of non-existence

Models, libraries, and services are released continuously. An agent that says "that does not exist" is often working from a stale catalogue. When the claim matters, open the source - OpenRouter, the relevant provider's site, the package registry and check directly.

Building without clarity

The biggest hidden cost is not technical. It is the token spend that comes from telling the agent to do something the human has not yet thought through. Spend a few minutes writing down what the project actually is before opening any tool. The PRD step is not a bureaucratic ceremony; it is the cheapest insurance available against expensive rework.

Skipping the file inspection

A build that happens entirely inside the chat window is a build the human cannot really see. Antigravity, VS Code, Cursor any of them exist so the human can open the files the agent created and read them. The .claude folder is hidden by default in the operating system but visible in any IDE; this is one of the reasons an IDE is part of the workflow rather than optional.

Closing Reflections

The session ends with a recap and a forward look. The recap re-traces the through-line: a tool pairing of Claude Code as the executing agent and Antigravity as the editor and backup, terminal basics with cd as the only must-know command, the chef-recipe-tools framing for skills and MCP servers, the importance of taste in choosing what to install and what to verify, and a live build that produced a working multi-model debate application from an empty folder in roughly two hours.

What learners should take away

- The workflow is reproducible. The mentor showed it on a project he had not built before, in front of an audience, and shipped a working version. The same workflow will produce a different project with the same fluency once practised.
- The judgment moments which models, which skills, which design system, which web search provider, when to switch agents are where the value of a human builder concentrates. The mechanics are increasingly the agent's job.
- Skills compound. A library of well-curated playbooks, accumulated over months, becomes a personal advantage that competitors without it cannot easily match.
- Antigravity is not a competitor to Claude Code in this workflow; it is a complement. The dual-engine setup is what made the live outage a footnote rather than a session-ending event.

What tomorrow promises

Day two of the sprint moves from intermediate to advanced. The mentor previews the topics he intends to cover: his personal advanced setup; ClaudeMem for memory management; a skill called Superpowers that he will explain in detail; bypass patterns for the permission prompts that interrupt long agent runs; and a different application built from scratch using a different curated skill set, to demonstrate that the workflow generalises beyond the Model Console example.

THE MENTOR'S PARTING MESSAGE

A good foundation has been laid. The exploration is the learner's responsibility now. Tomorrow's session will assume what was shown today and build on top of it. Convert the time zone, find the link in the LMS or email or WhatsApp or calendar, and show up.

Housekeeping from the host

The session closes with the program manager walking learners through where to find the recording, the transcript, and the post-read material. Everything lives inside the AI Masterclasses and Sprints space on the Outskill platform, accessed from the left sidebar after logging in. Recordings appear within roughly twenty-four hours, since the conferencing platform takes time to process them. Learners who cannot see the workspace are directed to check the email used for login, then to look for an invitation email including the spam folder and accept the workspace invite.

A note on the conversation that did not happen

Throughout the session, the mentor declined to engage with several categories of questions: which IDE is best, what about Bedrock, why this stack instead of that one, can you also show VS Code or Cursor or Windsurf in detail. His position was consistent. The session is about a system that works; the system can be transferred to other tools by the learner; the comparison shopping is a personal exploration, not a teaching moment. The discipline of saying no to tangential requests is what kept the session focused enough to fit a real live build into the time available.

Appendix: Quick Reference

Core slash commands

Command	Purpose
/login	Connect Claude Code to the user's Anthropic plan or API credits.
/usage	View current session and weekly usage. Check often.
/model	Switch between Opus, Sonnet, and other available models.
/skills	List currently available skills inside the project.
/mcp	List configured Model Context Protocol servers.
/plugin	Manage plugin installations from marketplaces.

The build, in one screen

Phase	Action
Frame	Decide what you are building. One sentence, written down.
Folder	Create a dedicated project folder. Open it in the editor.
Install	Add Claude Code extension from Anthropic. Log in. Check usage.
Curate	Import a vetted skills repository. Install only what fits this build.
Specify	Brainstorm with the agent. Lock model, mechanics, form factor, output.
Verify	Check any factual claims the agent makes against primary sources.
PRD	Generate the document. Read it in the file tree.
Design	Pick a system from SuperDesign. Merge it into the PRD.
Build	Switch to Sonnet. Scaffold. Code. Create .env. Paste keys yourself.
Secure	Confirm .gitignore excludes .env before any commit.
Run	Start the dev server. Test the core flow.
Improve	Identify the biggest failure mode. Fix it. Test again.
Ship	Push to GitHub with a clean README. Optionally deploy via Vercel or Netlify.
Recover	If Claude is down, switch to Antigravity Agent Manager. If tokens run out, pause or downgrade.

Credits and references

- **Mentor:** KVS Dileep Head of Generative AI Education, Outskill.
- **Host:** Kartik Program Manager, Outskill.
- **Series:** AI Masterclasses & Sprints by Outskill.

- **Day one focus:** Intermediate taste, judgment, the workflow scaffolding.
- **Day two focus:** Advanced the mentor's personal harness, memory management, orchestration patterns.
- **Tools used:** Claude Code (Anthropic), Antigravity (Google), OpenRouter, SuperDesign, GitHub, Vercel/Netlify (mentioned).
- **Skills source:** PM Skills repository by Pavel Hurin.
- **Reference site:** codingformarketers.com terminal basics and environment-variable primers.

End of summary.